Certifying the Judge: Amortized Human Validation of LLM Judges via Falsifiable Properties

Anonymous Authors[1]

Abstract

LLM judges are becoming essential for evaluating AI-generated code, tests, and formal specifications, but their reliability is usually established in the least scalable way: collecting new human ratings for every judge, prompt, model, and threshold. We challenge this validation loop by asking whether human judgment can be amortized through falsifiable behavioral properties. We propose a property-certification protocol for LLM judges of Lean 4 theorem and specification quality, where each judge is scored by four automatically testable conditions that don't require a gold reference and can work with any arbirary Lean file: identity, bug-monotonicity, specification-monotonicity, and repeat/prompt stability. These properties define a graded Trust Index, $\mathrm{TI} = (P_1P_2P_3P_4)^{1/4}$, intended not as a proof of correctness but as a measurable proxy for whether a judge behaves like a reliable evaluator.

On a validation panel of eight judges spanning frontier LLM prompts, weaker LLM baselines, edit-distance, and random controls, TI strongly predicts validation-set human alignment: a Spearman correlation coefficient of $\rho = 0.881$ against human judgements is achieved. Property-level analysis shows that monotonicity under removed specifications is the strongest single predictor of human agreement, followed by monotonicity under injected bugs, while identity and stability are weaker standalone signals. Applied to VERIBENCH, property certification provides a reproducible calibration protocol for scalable oversight of AI-generated tests, theorems, and formal specifications.

[1]**AUTHORERR: Missing \icmlaffiliation.** . **AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Agentic AI systems increasingly generate both code and the trust artifacts meant to evaluate that code: tests, specifications, and formal theorems. In classical autoformalization, the target theorem is supplied, so evaluation can often reduce to proof success. In agentic verification, the system must decide *what should be true* before it can prove anything. The Lean kernel can check a proof, but it cannot tell us whether the proposed theorem is complete, semantically aligned with the intended program, or merely a weak statement that happens to typecheck. As agents produce more candidate artifacts than humans can inspect, theorem-quality evaluation becomes a scalable-oversight problem (Bowman et al., 2022; Zhou et al., 2025; Khattab et al., 2024; Cheng et al., 2024).

The standard reliability check is per-judge human validation: collect labels, measure agreement between each judge and human ratings, and keep the judge if the correlation is high enough. That check is necessary, but it scales linearly with the number of judges, prompts, model updates, thresholds, and domains under consideration. Every new model family or prompt revision asks for another round of annotation, so direct human validation becomes untenable at agent-generation rates. Humans remain the anchor; the problem is that re-using humans from scratch for every judge wastes the expensive part of the process 1.

Validating LLM judges against humans is the standard reliability check but does not scale: every new judge, prompt, or model needs fresh annotations. Property certification **amortizes** this cost—gold references collected once, used to calibrate a four-property trust index (identity, bug-monotonicity, spec-monotonicity, repeat-stability), then deployed without further human annotation. In our setting, humans rate a validation set once, and the calibrated property suite extends that trust signal to new judges for free. The four properties are concrete: a judge should give maximal scores to identical artifacts, lower scores as semantic bugs accumulate, lower scores as theorem/test specifications are removed, and stable scores under prompt rewordings and semantics-preserving rewrites. On a panel of eight judges, the resulting variance-aware trust index predicts validation-set human Spearman correlation coefficients of 0.833 (avg labels), 0.905 (pass1),

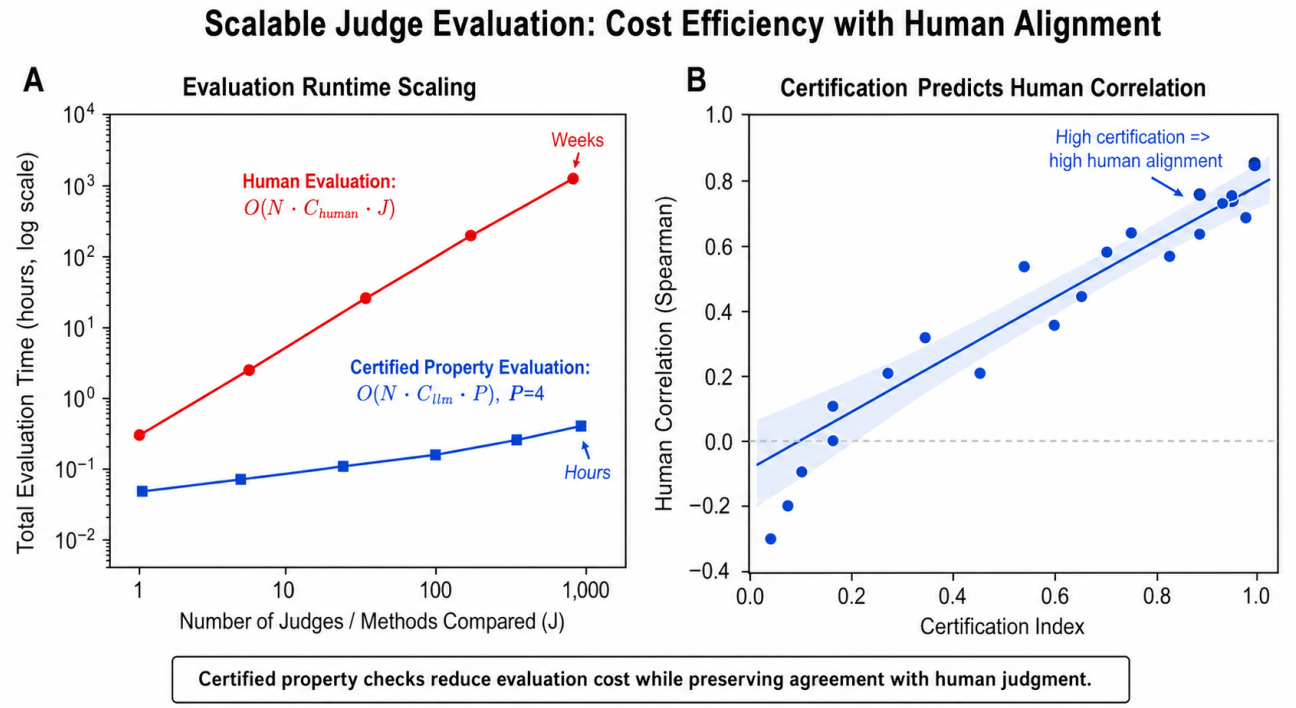


Figure 1: **Scalable judge evaluation via property certification.** Left: direct human validation cost scales with the number of judged methods and can become expensive at agentic throughput. Right: higher certification strength is associated with higher human alignment, supporting property certification as an amortized validation protocol.

and 0.738 (pass2); held-out test results are reported in a forthcoming revision.

Our **contributions** are:

1. **An amortized trust-validation protocol:** we formalize property certification as a one-time human calibration on a gold reference set, after which any new (LLM + prompt + n_{repeats} + aggregation) judge tuple can be validated by computing four falsifiable properties without further human annotation.

2. **A validation-set calibration result:** on a panel of 8 judges, a variance-aware trust index combining quality and stability predicts validation-set human Spearman correlation coefficients of 0.833 (avg), 0.905 (pass1), and 0.738 (pass2), with P3 weak-monotonicity and P2 bug-rank monotonicity as the strongest single predictors. Held-out test results are reported in a forthcoming revision.

3. **An application to VERIBENCH:** a calibrated judge ranks agentic methods consistently with humans, distinguishing feedback-driven approaches (DSPy ReAct: 0.615) from baseline prompting (0.470).

2. Related Work

LLM-as-a-Judge. LLMs are increasingly used as *automated evaluators* across open-ended tasks, often reaching strong agreement with human ratings—e.g., GPT-4 judges achieving $>80\%$ agreement with human preference labels in dialogue (Zheng et al., 2023). However, large-scale audits caution that reliability varies by task and setup, and stress the need for explicit validation before replacing human judgment (Kocmi et al., 2024; Li et al., 2024). Common biases include position and verbosity effects and sensitivity to prompt phrasing (Zheng et al., 2023; Li et al., 2024).

Improving judge reliability. To mitigate biases and instability, multi-agent debate frameworks such as *ChatEval* emulate panel-style review to yield more robust consensus scores (Chan et al., 2024). Structured prompting like *G-Eval* elicits rubric-driven, stepwise reasoning prior to scoring, improving agreement with human criteria on NLG tasks (Liu et al., 2023). Reliability can also be quantified via repeated scorings and internal consistency measures (e.g., Cronbach's α) to detect prompt/run sensitivity (Li et al., 2024).

Code and formal-spec evaluation. For programming, single-pass metrics like pass@k can be brittle or unavail-

able; early LLM-judge approaches such as *ICE-Score* highlighted difficulty distinguishing partial vs. full correctness from a one-shot review (Zhuo, 2024). *CodeJudge* operationalizes a two-stage "analyze-then-summarize" procedure with a taxonomy of code faults, significantly improving correlation with functional ground truth (Tong & Zhang, 2024). In formal verification/autoformalization, prior work typically provides fixed target theorems, reducing evaluation to proof success. In contrast, agentic systems must propose specifications, necessitating *semantic* judges. We adopt a certification protocol (akin to metamorphic testing) and verify that our LLM judge obeys *identity/consistency*, *monotonicity* with respect to injected bugs and specification omissions, and *prompt/rewrite stability*, directly answering recent calls for validated, trustworthy LLM evaluators in safety-critical domains (Kocmi et al., 2024; Li et al., 2024).

Overview of VERIBENCH. Concretely, VERIBENCH consists of 857 theorems divided into five subsets:

1. **HumanEval** – 387 theorems derived from 56 standard programming puzzles from Chen et al. (2021);
2. **EasySet** – 278 theorems extracted from 41 introductory logic and programming tasks;
3. **CSSet** – 68 theorems drawn from 10 classical data-structure and algorithm problems;
4. **SecuritySet** – 112 theorems taken from 28 examples of buffer overflows, privilege escalation, and race-condition labs based on real code;
5. **RealCodeSet** – 12 theorems extracted from 5 Python standard library programs, used to test model performance on production-grade code.

This design covers a spectrum of tasks, from simple correctness theorems to more complex invariants, algorithmic properties, and real-world code verification. See the ICML AI4Math workshop poster for VERIBENCH for additional context (VeriBench).

Evaluation Consistency Recent work has shown that unconstrained chain-of-thought reasoning can reduce evaluation consistency, and that simple comparative prompting often outperforms complex multi-step pipelines (Chen et al., 2026). We interpret these findings as evidence that stability, rather than reasoning complexity, is the primary driver of reliable grading. Our work formalizes this intuition by introducing certifiable structural properties—identity, monotonicity, and reliability—that predict agreement with human judgments.

3. Problem Setup: Judging Lean Theorem/Specification Quality

Task. Each benchmark item contains a natural-language problem statement, a gold Lean 4 artifact g, and a candidate artifact x produced by an AI agent. The artifact may include code, tests or specification scaffolds, and a target theorem/specification statement. Our goal is to assign a score $J(x, g) \in [0, 1]$ reflecting how well the candidate captures the intended formal specification represented by the gold artifact.

Why this is nontrivial. Lean can check whether a concrete proof script type-checks, but type-checking alone does not determine whether x formalizes the same intended property as g. A candidate may compile while being underspecified, over-constrained, vacuously true, or semantically shifted. Thus, exact automatic scoring of theorem/specification quality is unavailable in general.

Judge model. We therefore use an LLM judge parameterized by

$$\theta = (\text{model}, \text{prompt}, n_{\text{repeats}}, \text{aggregation}),$$

which maps a pair (x, g) to a score $J_\theta(x, g) \in [0, 1]$. The central question is not only how to build such a judge, but when it is trustworthy enough to rank systems. Rather than collecting new human labels for every θ, Section 4 tests whether J_θ satisfies falsifiable behavioral constraints under controlled perturbations.

4. Property Certification Protocol

4.1. Calibration Protocol

We frame property certification as a calibration protocol. **One-time setup:** humans rate a labeled subset of N gold artifacts under fixed rubrics; here, $n = 72$ aligned candidate–gold pairs are rated by one annotator under two rubrics, R1 and R2. **Per-judge, no human in the loop:** for each new judge tuple, compute property scores on scripted perturbations of Lean artifacts, then aggregate them in either arithmetic (AM) or geometric (GM) form:

$$\mathrm{TI}_f(\mathbf{v}, \mathbf{w}) = f^{-1}\Big(\sum_{i=1}^{d} w_i\, f(v_i)\Big), \quad w_i \geq 0,\ \sum_i w_i = 1,\ v_i \in (0, 1].$$

$$\text{AM: } f(x) = x \ \Rightarrow\ \mathrm{TI}_{\mathrm{AM}} = \sum_i w_i v_i, \quad \text{GM: } f(x) = \log x \ \Rightarrow\ \mathrm{TI}_{\mathrm{GM}} = \mathrm{e}$$

Shorthand labels (all in $[0, 1]$): ident (P1: identity/consistency), mono (P2: bug monotonicity), weak (P3: specification-ablation monotonicity), stab (P4: repeat/prompt stability), flip = repeat-level pass/fail disagreement rate.

Indices used in this paper:

$$\text{v1} = 0.30\,\text{ident} + 0.30\,\text{mono} + 0.25\,\text{weak} + 0.15\,\text{stab},$$

$$\text{v2} = 0.20\,\text{ident} + 0.20\,\text{mono} + 0.15\,\text{weak} + 0.10\,\text{stab} + 0.20\,\text{anti_string} + 0.15\,\text{subtle_bug}.$$

Primary metric in this revision:

$$\text{TI}_{\text{var}} = \exp\Big(\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{\text{VarCorr}}) + \tfrac{1}{3}\log(1-\widetilde{\text{VarScore}})\Big),$$

where C_f is a per-judge quality term computed from property behavior (P1–P4 diagnostics), and $\widetilde{\text{VarCorr}}$, $\widetilde{\text{VarScore}}$ are normalized variance penalties. **Calibration claim:** trust-index scores on the validation set predict each judge's validation-set human Spearman. **Deployment:** any new (LLM + prompt + n_{repeats} + aggregation) tuple is evaluated for trust by computing the same chosen trust-index form on the existing gold reference set, with no additional human annotation. Thus P1–P4 are the four calibration targets for the trust index, not binary gates: they grade how strongly a judge preserves identity, monotone degradation, and repeat stability on controlled perturbations.

Why is an LLM judge needed for theorem-quality scoring? Evaluating AI-generated theorems/specifications requires a method to assess their quality, as proof assistants can verify a proof's correctness but not whether the agent proved the correct theorem. This is challenging because proving the logical equivalence of two arbitrary theorem statements is generally undecidable, and an agent may generate theorems that are semantically valid but syntactically different or incomparable with the gold reference specification/theorem (e.g., by splitting a theorem into different parts).

Our goal is to calibrate the trustworthiness of an LLM judge for the task of formal specification/theorem generation. This is akin to scoring the quality of a conjecture. We operationalize this by requiring the judge to compare a candidate Lean 4 artifact (implementation, tests, theorems) against its gold-standard Lean 4 reference. We provide the full set of Lean 4 artifacts because theorems might not make sense in isolation. The judge returns a discrete score $s \in \{0, 1, \ldots, 10\}$ for correctness, completeness, and semantic alignment, which we later normalize to $[0, 1]$. We use this score as a metric signal for evaluating the specifications generated by the agents.

To be considered trustworthy, the judge should score well on the following falsifiable calibration targets. They provide graded evidence that the judge reacts logically to structured perturbations.

P1 — Identity & Consistency. The judge must be idempotent at the maximum score when comparing an artifact to itself. For any reference X, it must hold that $s(X, X) = 1$.

P2 — Monotonicity to Bugs. The judge's score must decrease as semantic bugs are cumulatively introduced. Let B_1 and B_2 be sets of bugs such that $B_1 \subset B_2$. For candidates C_{B_1} and C_{B_2} containing precisely these bug sets, the expected scores must satisfy $\mathbb{E}[s(C_{B_1})] > \mathbb{E}[s(C_{B_2})]$.

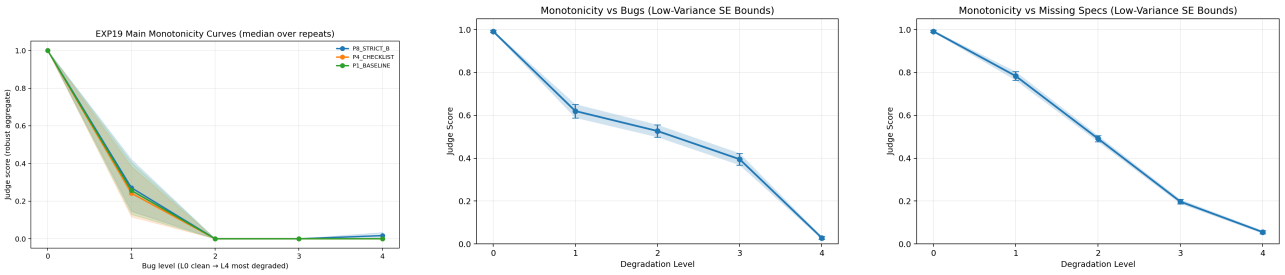
P3 — Monotonicity to Missing Specs. The judge's score must decrease as specification elements are removed. Let S_1 and S_2 be sets of specifications (theorems and tests) such that $S_2 \subset S_1$. For candidates C_{S_1} and C_{S_2} containing precisely these specification sets, the expected scores must satisfy $\mathbb{E}[s(C_{S_1})] > \mathbb{E}[s(C_{S_2})]$.

P4 — Prompt Stability & Semantic Invariance. The judge should produce stable scores under repeated calls and preserve judgments under semantics-preserving rewrites. For artifact pairs and fixed judge instructions, repeated-call variance should remain low and prompt-pair rank agreement should remain high.

We verify monotonicity by computing rank correlations (Kendall's τ, Spearman's ρ) between (k, m) and judged scores, and by reporting pairwise ordering accuracy (AUC) that "worse" artifacts receive lower scores (Tong & Zhang, 2024; Zhuo, 2024).

Decision-threshold calibration (Exp13). After validating score-level behavior (P1–P4), we calibrate the binary operating point used for deployment decisions. For threshold $t \in [0, 1]$, we define PASS iff $\bar{s} \geq t$, where $\bar{s}$ is the mean over repeated calls for the same pair. We sweep $t \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85\}$ and report: (i) *flip rate* (fraction of items whose repeat-level pass/fail decisions disagree), (ii) *pass rate* (coverage), and (iii) ranking diagnostics (Spearman bug-correlation) to verify that calibration affects classification stability, not score ordering. All reported threshold metrics include 95% bootstrap confidence intervals over files.

Establishing trust in the LLM judge. To ensure trustworthiness of the LLM judge in evaluating agentic autoformalization, we systematically validate key properties the judge must satisfy in Figure 2: (1) scores decrease consistently as more theorems and tests are omitted, (2) scores similarly degrade when more implementation bugs are introduced, and (3) semantically preserving edits keep scores high. In our latest GPT-5.3-Codex API sanity run ($n_{\text{files}} = 184$, $n_{\text{repeats}} = 3$), monotonic trends remain strong (bugs: Spearman $\rho = -0.808$, Pearson $r = -0.687$; missing specs: Spearman $\rho = -0.899$, Pearson $r = -0.892$), supporting use of the judge as a semantically meaningful proxy for correctness/completeness alignment across VERIBENCH subsets (Table 6) (Li et al., 2024; Kocmi et al., 2024).



(a) Low-variance monotonicity (main view) (b) P2: Monotonicity vs. Bugs (Mean ± SE) (c) P3: Monotonicity vs. Missing Specs (Mean ± SE)

Figure 2: **LLM Judge Trustworthiness Sanity Checks.** VeriBench-wide GPT-5.3-Codex API run ($n_{\text{files}} = 184$, $n_{\text{repeats}} = 3$). **(a)** Low-variance rendering (Fig14-style) emphasizes smooth non-increasing score behavior under controlled degradations. **(b)** P2 bug-ladder trend (2025 Fig2-style) remains directionally correct with local reversals (Pearson $r = -0.687$, Spearman $\rho = -0.808$, weak monotonicity 52.2%). **(c)** P3 missing-spec trend (2025 Fig2-style) is stronger and more stable (Pearson $r = -0.892$, Spearman $\rho = -0.899$, weak monotonicity 89.1%). Strict P1 identity $s(X, X)$ and P4 rewrite invariance are reported separately in Table 1.

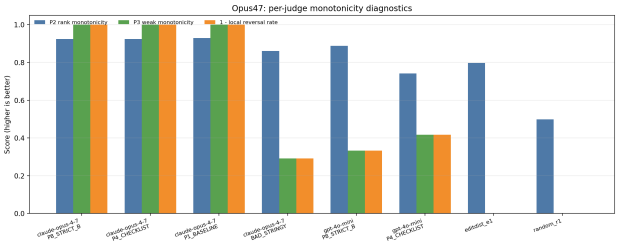


Figure 3: **Opus 4.7 per-judge monotonicity diagnostics.** Bars report P2 bug-rank monotonicity, P3 weak monotonicity, and $1-$local-reversal rate by judge. Core Opus prompts remain near the top of all three monotonicity diagnostics; stress controls are lower.

5. Experiments

5.1. Data Setup: VERIBENCH, Alignment, and Human Labels

All experiments are built on VERIBENCH, spanning five subsets (Easy, CS, RealCode, HumanEval, Security; Section 2). For judge-validation experiments, we use an aligned validation panel of $n = 72$ candidate–gold pairs formed from 24 problems with 3 agent generations per problem (`agent0`, `agent1`, `agent2`). Each aligned row is scored by humans under two passes (pass1/pass2) on a 0–5 rubric, and we report pass1, pass2, and averaged labels.

This setup separates *data construction* from *judge calibration*: the same aligned 72-row panel is reused across judges so trends reflect judge behavior, not row-selection changes. Human-pass dispersion on this aligned panel is non-trivial (Appendix Fig. 17), motivating our practice of reporting all three label views rather than a single pass.

5.2. Downstream Agent Ranking on VERIBENCH

Table 6 presents the normalized average scores across different splits of VERIBENCH: Easy Set (Easy), CS Set (CS), Real Python Code (Real), HumanEval Set (HE), and Security Set (Sec). As expected, the baseline prompting approach achieves the lowest overall normalized score (0.470), reflecting its limited capability in autonomously identifying comprehensive theorem properties. It also gives us further confidence in our judge. Conversely, agentic methods leveraging iterative self-debugging and self-improvement demonstrate significant enhancements. Specifically, the DSPy ReAct agent, which incorporates feedback-driven self-debugging, achieves the highest overall average score of 0.615. The Trace++ agent, employing both self-debugging and self-improvement mechanisms, also shows notable improvement, attaining an average score of 0.588. These results underscore the effectiveness of agentic, feedback-driven approaches in enhancing theorem-generation quality compared to non-agentic baselines (Khattab et al., 2024; Cheng et al., 2024; Chen et al., 2021).

5.3. Property sanity checks on GPT-5.3-Codex

We added a new property-calibration run in `codex_api_gpt53_r3_20260418_230027` under the experiment results directory. This run uses GPT-5.3-Codex in API judge mode with three repeated judge calls per pair ($n_{\text{repeats}} = 3$), over 184 Lean files (736 identity rows and 920 rows each for bug/missing ladders).

What the data shows. Monotonicity with missing specifications is strong ($\rho = -0.899$, $r = -0.892$, weak monotonicity 89.1%), indicating stable penalty as theorem content is removed. Bug monotonicity is directionally correct but noisier ($\rho = -0.808$, $r = -0.687$, weak monotonicity 52.2%), suggesting some bug ladders induce local

Identity Variant	Mean Score	Mean Within-Item Std
RENAME	0.936	0.005
REORDER	0.849	0.029
STYLE	0.982	0.001
ALPHA	0.399	0.096

Table 1: **P4 semantic-invariance breakdown for the Codex API run (not P1 identity).** Variant-wise score behavior under semantics-preserving rewrites. The weakest mode is aggressive ALPHA-renaming.

Threshold	Flip Rate (95% CI)	Pass Rate (95% CI)
0.60	0.058 $[0.048, 0.069]$	0.583 $[0.559, 0.608]$
0.70	0.049 $[0.040, 0.059]$	0.559 $[0.537, 0.581]$
0.80	0.051 $[0.042, 0.061]$	0.544 $[0.523, 0.566]$
0.85	**0.046** [**0.037**, **0.057**]	0.536 $[0.515, 0.557]$

Table 2: **Threshold calibration summary (Exp13, full 176-file run).** Calibration yields a modest but consistent reduction in decision flips at stricter thresholds, with moderate coverage loss. Ranking behavior is unchanged across thresholds ($\rho_{\text{bug}} = -0.790$ at all t).

score reversals. Strict identity (P1) is reported only on exact self-pairs $s(X, X)$; rewrite robustness is treated separately under P4. Under P4, semantic invariance is high for STYLE/RENAME but weak for aggressive ALPHA-renaming, which drives most of the invariance gap.

Interpretation. The judge reliably captures coarse semantic degradation (especially missing-spec ablations), while lexical/format-level robustness is transformation-dependent. Practically, this motivates canonicalization of binder names and stronger structure-aware prompting before using identity as a hard trust gate. The run metadata also notes appended datapoints from 34 added tasks scored in local mode, so direct cross-run comparisons should be treated as provisional until fully re-run under one scoring mode.

5.4. TI predicts validation-set human alignment

We add a dedicated P4 run in `stability_gpt-53-codex_full_20260420_185401` using GPT-5.3-Codex in API mode over 184 files, 4 prompt templates (`P1_BASELINE`, `P2_FORMAL`, `P4_CHECKLIST`, `P8_STRICT_B`), and 5 repeats per prompt ($n_{\text{cells}} = 3680$). With no prompt changes (same template, repeated calls), run-to-run noise is low (mean_repeat_std $= 0.0330$). Under instruction-equivalent prompt rewordings, cross-prompt score dispersion is moderate (0.0782), while overall internal consistency remains high (Cronbach's $\alpha = 0.9416$). Prompt pairs preserve ranking strongly (minimum Spearman $\rho = 0.9550$), but binary agreement is weaker near threshold (flip rate $= 31.3\%$, minimum Cohen's $\kappa = 0.4540$). Semantic invariance gaps are smallest for STYLE (0.0042), moderate for RENAME (0.1021) and REORDER (0.2128), and largest for ALPHA (0.6325), indicating that alpha-conversion handling is the dominant robustness bottleneck.

5.5. Threshold Calibration: Stability vs Coverage

We run Exp13 to choose a binary pass/fail threshold for deployment from continuous judge scores. On 176 files (`veribench_dataset_newest`) with GPT-5.3-Codex (`P8_STRICT_B`, $n_{\text{repeats}} = 3$, 1000 bootstrap resamples), changing t mostly affects decision behavior, not ranking behavior. Across $t \in [0.60, 0.85]$, bug-ladder Spearman is essentially unchanged ($\rho = -0.790$, 95% CI $[-0.810, -0.769]$). As t increases, flip rate improves (0.058 $\rightarrow$ 0.046) while pass rate drops (0.583 $\rightarrow$ 0.536). Using our rule (minimize flip rate, then prefer pass rate near 0.5), we select $t = 0.85$.

5.6. Robustness: Opus / expanded judge panel / R9 median

We ran a final multi-judge validation-set human-alignment suite over 72 aligned candidate–gold pairs (24 problems $\times$ 3 candidates), using 8 judges and 3 validation label sets: pass1, pass2, and their average. Judge-side uncertainty used 10,000 bootstrap iterations and 20,000 permutation iterations, with 3 repeated judge calls per pair and threshold $t = 0.85$.

Experiment design. **Exp16** asks whether a variance-aware trust index predicts validation-set human Spearman *across judges*: for each judge, compute quality and stability diagnostics and correlate their aggregate index with that judge's validation-set human Spearman. **Exp17** decomposes this relation by property (P1–P4), and reports per-property correlations, partial correlations, and multivariate regression. Our primary metric is

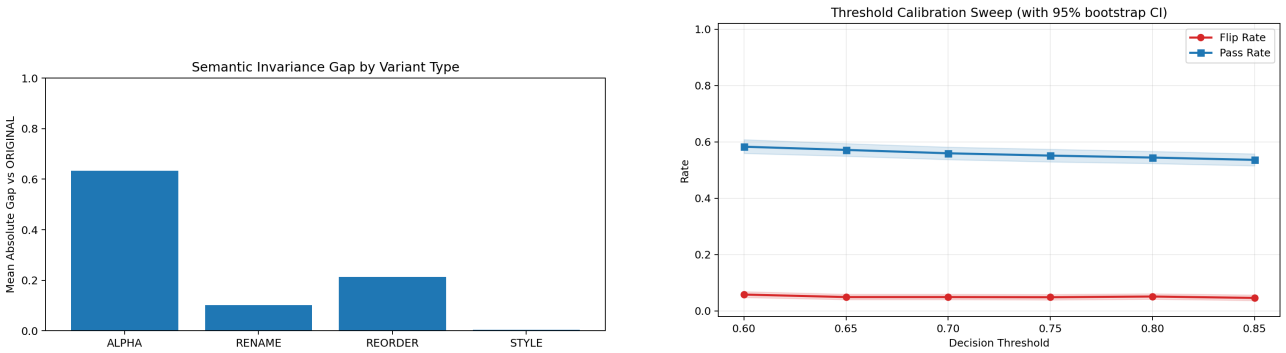
$$\text{TI}_{\text{var}} = \exp\Big(\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{V_f}) + \tfrac{1}{3}\log(1-\widetilde{V_s})\Big),$$

where C_f is mean per-file monotonicity quality, V_f is variance of per-file monotonicity correlation, and V_s is mean repeat-score variance.

Each point in Exp16 is *one judge tuple* (model + prompt/rubric + repeats + aggregation + threshold). For judge j, let h_i be the human rating and $\hat{s}_{ij}$ be that judge's score on item i, over the same aligned item set $i = 1, \ldots, N$. The y-axis value is

$$y_j = \rho_s\big((h_i)_{i=1}^N,\ (\hat{s}_{ij})_{i=1}^N\big),$$

i.e., **per-judge human agreement**, not a raw score level. The x-axis is that judge's trust index $\text{TI}_{\text{var},j}$, computed from



(a) P4 semantic invariance gaps by variant type.

(b) Decision calibration curve from Exp13.

Figure 4: **Stability diagnostics and deployment calibration.** Left: mean absolute gap vs ORIGINAL across semantics-preserving variants; ALPHA remains the dominant invariance failure mode. Right: as threshold increases, pass rate declines while flip rate improves, reaching minimum flip rate at $t = 0.85$.

Label Set	Spearman Corr. Coef. ($\mathrm{TI}_{\mathrm{var}} \rightarrow$ human)
pass1	0.905
pass2	0.738
avg	0.833

Table 3: **Exp16 validation trend with the variance-aware trust index.** Each row uses all 8 judges from the Codex panel. Spearman correlation coefficients are reported between judge-level $\mathrm{TI}_{\mathrm{var}}$ and judge-level human alignment.

perturbation properties. So Exp16 tests whether judges that satisfy certification properties better also *agree with humans better as evaluators*.

Why we do not regress TI on raw human scores. Raw human scores are item-level absolute ratings and depend on item difficulty/content. Our target is judge reliability as an evaluator, which is judge-level agreement with humans across items. Therefore the evaluation object is $\rho_s(\text{Human}, \text{Judge})$ per judge, and the main claim is: high trust-index judges tend to have higher human-agreement coefficients.

Validation-set result (judge-level trend). The variance-aware trust index predicts validation-set human Spearman across all three validation label sets (Table 3): 0.833 (avg), 0.905 (pass1), and 0.738 (pass2). Thus, trust defined as quality + stability is strongly aligned with human judgment at the judge level. Judges that are simultaneously high-certification, monotonic, and stable are the ones that align best with humans, and this pattern persists as we scale to larger judge panels.

Human-label verification (pass consistency). To quantify label-pass dispersion directly on the same aligned

Protocol	Fit Spearman Corr. Coef.	Test Spearman Cor
Fit on pass1 $\rightarrow$ test on pass2	0.881	0.810
Fit on pass2 $\rightarrow$ test on pass1	0.857	0.786

Table 4: **Cross-pass generalization (no-overfitting check).** We fit trust-index weights on one human pass and report on the other pass with no re-tuning. Test-set Spearman correlation coefficients remain > 0.7 in both directions.

Model	pass1	pass2	avg
Codex	0.905	0.738	0.833
Claude Opus	0.810	0.595	0.643
Qwen3-Coder	0.476	0.905	0.905
Leanstral	-0.119	-0.048	-0.095

Table 5: **Cross-model trends for $\mathrm{TI}_{\mathrm{var}}$.** Entries are Spearman($\mathrm{TI}_{\mathrm{var}}$, per-judge human Spearman) across judges for each validation label set.

$n = 72$ pairs, we compute pass1-vs-pass2 absolute error and distribution divergence. The inter-pass MAE is 0.778 on the 0–5 rubric scale (median absolute difference $= 1.0$); distribution-level divergence is moderate. This confirms non-trivial pass drift while still supporting meaningful judge-level trend analysis.

Judge-level effect sizes. Under averaged validation labels, top TI-ranked judges are GPT-5.3-Codex prompts: `P4_CHECKLIST` ($r_s = 0.480$, $p = 2.0 \times 10^{-5}$), `P1_BASELINE` ($r_s = 0.470$, $p = 3.1 \times 10^{-5}$), `P8_STRICT_B` ($r_s = 0.356$, $p = 0.0022$). In the Qwen replication, the strongest pass2-aligned prompts are `Qwen|BAD_STRINGY` ($r_s \approx 0.304$), `Qwen|P4_CHECKLIST` ($r_s \approx 0.280$), and `Qwen|P1_BASELINE` ($r_s \approx 0.194$).

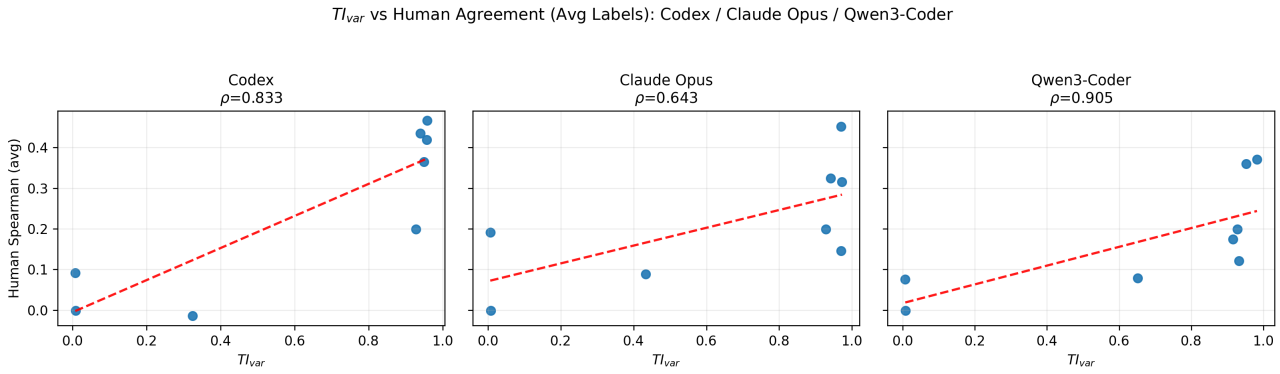


Figure 5: $\mathrm{TI}_{\mathrm{var}}$ **vs human agreement (avg labels).** Each panel plots per-judge $\mathrm{TI}_{\mathrm{var}}$ against per-judge human Spearman for one model family (Codex, Claude Opus, Qwen3-Coder), with a dashed linear trend line.

Lower-TI baselines are markedly weaker: `gpt-4o-mini|P8_STRICT_B` ($r_s = 0.037$, $p = 0.760$), `gpt-4o-mini|P4_CHECKLIST` ($r_s = 0.093$, $p = 0.435$), `random` ($r_s \approx 0$, $p = 0.995$).

Property-level interpretation (Exp17). For averaged labels, per-property correlations identify **P3 weak monotonicity** as strongest (Spearman 0.747, $p = 0.043$; Pearson 0.744, $p = 0.048$), with **P2 bug-rank monotonicity** next (Spearman 0.714, $p = 0.056$; Pearson 0.636, $p = 0.037$). P1 identity and P4 stability are not strong standalone predictors at this $n_{\text{judges}} = 8$ scale. Across pass1/pass2/avg, P3 is consistently top-ranked, indicating that semantic monotone behavior under controlled degradations is the most stable predictor of validation-set human Spearman in this study.

5.7. Failure modes: alpha-renaming and local reversals

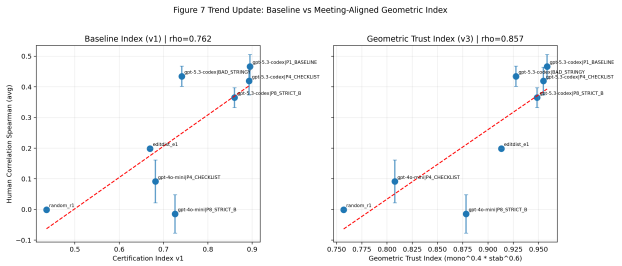
We replicate the certification protocol using Claude Sonnet 4 as the judge model on a preliminary subset of three VeriBench files (one each from CS, Easy, and HumanEval), with five degradation levels and three repeated calls per pair. Identity (P1) remains perfect ($\mu = 1.000$, $\sigma = 0.000$), confirming that self-comparison yields maximal scores regardless of judge model. Bug monotonicity (P2) shows a clear negative trend (Pearson $r = -0.717$, Spearman $\rho = -0.674$, weak monotonicity 66.7%), and specification ablation (P3) is directionally correct (Pearson $r = -0.560$, Spearman $\rho = -0.402$). Prompt stability (P4) yields a pooled within-item standard deviation of 0.088, comparable to the Codex run. These preliminary cross-model results suggest that the certification properties are not specific to a single LLM family; a full-scale Claude run across all VeriBench files is planned.

Claude Opus 4.7 replication (full Exp16 panel). On the full 8-judge Exp16 validation panel, we re-run the same analysis with Claude Opus 4.7 and apply the same variance-aware index $\mathrm{TI}_{\mathrm{var}}$. The resulting Spearman correlation coefficients are pass1 $= 0.810$, avg $= 0.643$, pass2 $= 0.595$. This supports cross-model transfer of the same quality+stability index design, even when absolute judge-level correlations differ by label pass.

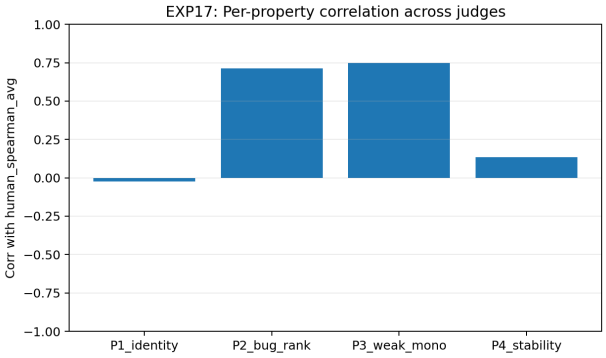
Qwen3-Coder replication (full Exp16 panel). We additionally run a full 8-judge open-weights replication with `Qwen/Qwen3-Coder-30B-A3B-Instruct` as the primary judge family (plus `gpt-4o-mini`, edit-distance, and random controls), with the same $n_{\text{repeats}} = 3$, median aggregation, and $t = 0.85$ protocol. On pass2 labels, baseline cert-v2 yields $\rho_s = 0.833$, and a legacy geometric baseline ($\mathrm{TI}_{\mathrm{geom}} = \mathrm{mono}^{0.4} \cdot \mathrm{stab}^{0.6}$) improves to $\rho_s = 0.857$. Pass1 and avg trends are weaker/noisier (-0.190, 0.500), so we treat Qwen as cross-model robustness evidence rather than as a standalone primary claim.

Expanded-panel scaling (13 judges). To test whether the certification trend persists beyond the 8-judge panel, we run an expanded 13-judge analysis (additional Codex prompt variants and controls; same 72 aligned rows, $t = 0.85$). Using the same geometric-index family, the trend remains strong across all label sets: pass1 $\rho = 0.764$, pass2 $\rho = 0.753$, avg $\rho = 0.769$. This supports the claim that property-based trust indexing scales with judge-panel diversity, not only with a minimal panel.

Generalization to a larger judge panel. Using a legacy geometric-index variant on an expanded *13-judge* panel (extra Codex prompt variants and baselines, same 72 aligned rows, $t = 0.85$), we still observe > 0.7 trends across label sets: pass1 $\rho = 0.764$, pass2 $\rho = 0.753$, avg $\rho = 0.769$. This indicates that the geometric certification trend persists as judge-panel diversity increases.



(a) Exp16 (avg labels): TI score vs. validation-set human Spearman.



(b) Exp17 (avg labels): per-property correlation with validation-set human Spearman.

Figure 6: **Final robust validation-set human-alignment results.** Left: validation-set scaling trend (Exp16) using averaged validation labels; each dot is one judge and y-axis is that judge's item-level Spearman agreement with humans. Right: validation-set per-property correlation (Exp17), showing P3 (weak monotonicity) and P2 (bug-rank monotonicity) as the strongest positive predictors of validation-set human Spearman.

Flip-rate and per-judge monotonicity diagnostics (Opus 4.7). At $t = 0.85$, repeat-call decision instability is low for core Opus prompts: `P4_CHECKLIST` and `P1_BASELINE` both have flip rate 0.000, and `P8_STRICT_B` has 0.0069. Per-judge monotonicity diagnostics are also strongest for these core prompts, while stress controls (BAD_STRINGY, edit-distance, random) show larger local-reversal behavior. This section focuses on *within-judge monotonicity* (bug level vs score), which is strong for Opus: `P8_STRICT_B` Spearman $= -0.849$, `P4_CHECKLIST` Spearman $= -0.849$, and `P1_BASELINE` Spearman $= -0.858$ (median aggregation). For visualization, we report mean $\pm$ SE error bars as a repeat-noise proxy: for each item, compute the repeat-call standard deviation across K calls, convert to $s_i/\sqrt{K}$, and average across items to obtain the plotted half-width; these are not 95% CIs, and the underlying Spearman ranking statistics are unchanged.

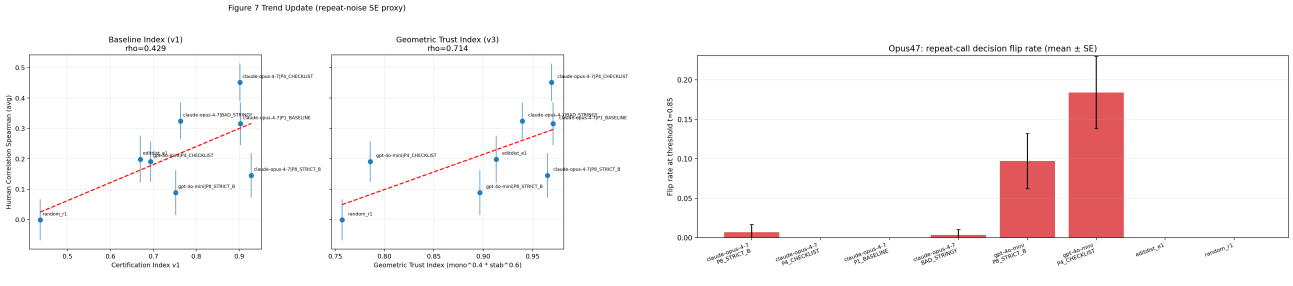
		Split			
Agent	**Eval**	**Easy**	**CS**	**Real**	**HE**
Baseline Prompting	Normalized	0.580	0.690	0.472	0.410
DSPy ReAct	Normalized	0.659	0.754	0.580	0.650
Trace+ (Self-Debug)	Normalized	0.342	0.680	0.592	0.573
Trace++ (Self-Improve)	Normalized	0.620	0.756	0.504	0.645

Table 6: **Theorem Quality Scores.** Average normalized scores in VeriBench split: Easy Set (Easy), CS Set (CS), Real Python Code (Real), HumanEval Set (HE), Security Set (Sec). All rows use Claude 3.7 as the agent model and as the LLM judge. Trace+ denotes the trace self-debug agent and Trace++ the self-debug and self-improve agent. The DSPy ReAct agent only performs self-debug. The LLM judge issues 5 scoring calls per file pair and returns an integer rubric score in $\{0, 1, \ldots, 10\}$. Normalized scores are computed by dividing by the maximum score (10) and averaging across all examples. The Avg Norm Score shows the overall average over the entire benchmark.

5.8. Downstream VeriBench agent ranking

6. Limitations and Held-Out Test Protocol

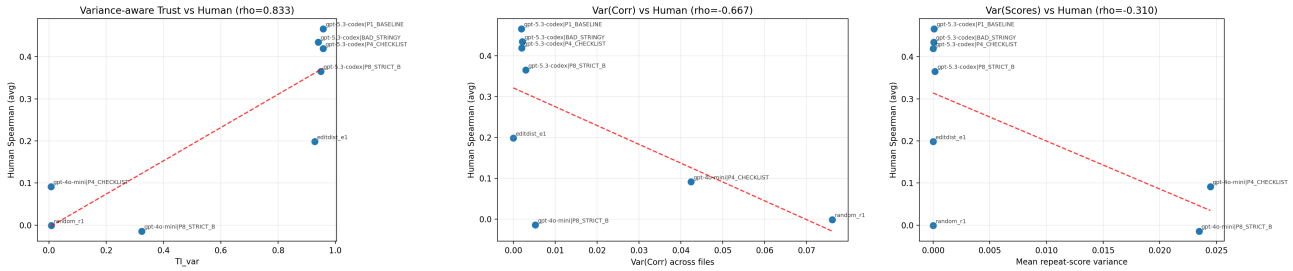
We study an LLM judge that *compares each candidate artifact against its task's gold reference* for Lean 4 code verification and a framework to increase confidence in it. The judge satisfies falsifiable score-level properties (identity and monotonicity) and, with Exp13, we additionally calibrate its binary decision boundary using full-dataset threshold



(a) Opus 4.7 Exp16 trend (avg validation labels): legacy baseline vs geometric variant ($\rho = 0.143 \rightarrow 0.714$); our primary index is $\mathrm{TI}_{\mathrm{var}}$.

(b) Opus 4.7 repeat-call decision flip rates at $t = 0.85$ (mean $\pm$ SE).

Figure 7: **Claude Opus 4.7 validation diagnostics.** Left: geometric aggregation strengthens the judge-level certification→human trend. Right: binary decision instability remains low for core Opus prompts.



(a) Variance-aware trust index vs. human

(b) Var(Corr) vs. human

(c) Var(Scores) vs. human

Figure 8: **Variance-aware trust framing across judges (Codex, avg labels).** Left: higher combined trust index corresponds to better human agreement. Middle/right: lower variance of per-file correlation and lower repeat-score variance tend to align with higher human agreement. This makes the stability requirement explicit alongside monotonicity and supports the claim that high-certified monotonic + stable judges align better with humans.

sweeps with uncertainty estimates. Our data supports a clear separation: score ranking remains stable under threshold changes, while binary stability and coverage trade off in a tunable way. In experiments, calibrated TI judging produces informative method rankings and tracks engineering signals (Pass@k, unit-test accuracy); Easy and HumanEval show the strongest agreement, while Security and RealCode remain hardest (Chen et al., 2021).

6.1. Validation vs. test protocol

The results in Section 5.6 are validation-set numbers. The same human ratings (single annotator under rubrics R1 and R2; $n = 72$ aligned pairs) were used to select: (i) trust-index formulation and weighting ($\mathrm{TI}_{\mathrm{var}}$ vs. legacy variants); (ii) repeat aggregation (median vs. mean); (iii) decision threshold $t = 0.85$ (Exp13); (iv) prompt panel composition; and then to report $\rho = 0.881$ against. Calibration-side optimization and reporting share the same labels.

We treat this as a calibration finding rather than a generalization claim. A held-out TEST protocol is in progress with three differences from the validation set: (a) independent annotators (multiple humans, including Elyas as a second annotator) under rubric R2 frozen; (b) held-out judges from a model family unseen during validation (e.g., Gemini, DeepSeek, or a new prompt template); (c) held-out items from VERIBENCH problems not in the $n = 72$ calibration set.

All test-set choices (judges, items, prompt panel, TI weighting) will be frozen in a separate `PROTOCOL_test.md` BEFORE any test invocation, with a single test run reported as-is regardless of outcome. We will report the test ρ in a forthcoming revision; if it differs materially from the validation $\rho = 0.881$, we treat the discrepancy as a methodological finding rather than a result to retune.

6.2. Finite perturbations, specification-relative scoring, and release details

We treat TI as a graded trust signal rather than a binary pass/-fail gate. Individual properties are necessary conditions on the perturbations we test (a judge that scores $X = X$ at less than 1.0 has, by definition, failed identity), but no finite property suite can certify all possible judge failures. TI is

multiplicative, so a single low-scoring property collapses the index, but it does not impose a hard threshold. The empirical question we answer is whether higher-TI judges are more validation-human-aligned in expectation; the answer ($\rho = 0.881$ across 8 judges) is supportive but graded, and generalization to held-out humans and judges is reported separately.

Limitations. The judge is a *proxy*: scores are prompt/rubric-sensitive and do not constitute kernel-level proof checking; they provide evidence against a rigorous yet necessarily incomplete gold specification since no complete mechanical checks of specification equivalence exist (Rice, 1953). Our certification relies on a finite perturbation suite, so untested failure modes may persist. Specifications that are exotic to this dataset may be correct but receive incorrect scores. Even in stronger runs, alpha-renaming remains the weakest invariance mode (lower mean / larger gap than STYLE and often RENAME), making binder-rename robustness a clear future-work target. Calibration improvements are modest in absolute magnitude and do not by themselves establish external validity. The validation set has one annotator under two rubrics (R1/R2) on $n = 72$ aligned pairs and $n = 8$ judges; Elyas is planned as an independent second annotator under the frozen R2 rubric. The current validation-set human-alignment results should be read as calibration evidence because judge, prompt, aggregation, threshold, and reporting choices were developed with this label set in view; a locked held-out human-label test remains in progress before claiming broad out-of-sample amortization. BAD_STRINGY, an intentionally bad baseline, achieves moderate validation-set human Spearman (≈ 0.45) while failing weak monotonicity (12.5%). This is a feature of the calibration protocol, not a bug: aggregate validation-set human Spearman alone does not detect this judge's monotonicity failure, but TI does. Future work will expand independent raters and problem coverage, strengthen semantic equivalence handling (candidate $\Rightarrow$ gold postcondition checks), add adversarial judge stress tests, and introduce standardized "judge cards" for reproducible calibrated deployment (Zhou et al., 2025). Compositional certification—extending local property checks to bounded edit sequences via edit-graph induction—is a natural future-work direction, but not a claimed result here.

7. Conclusion

We present an amortized human-validation protocol for LLM judges based on falsifiable properties. Rather than re-labeling every new judge configuration, we calibrate once with human labels and then score new judges via property diagnostics. Our main empirical result is that this property-based trust index tracks judge-level human alignment on the validation panel, with strong signals from monotonicity and stability. The key open step is locked held-out testing with independent annotators and unseen judge families.

References

Ahuja, . et al. Improver: Lean-aware chain-of-states prompting for proof rewriting and repair. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

Aniva, . et al. Pantograph: Programmatic access to lean 4 for learning-based proof search. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

Bach, R. et al. A systematic literature review of user trust in AI-enabled systems: An HCI perspective. *International Journal of Human–Computer Interaction*, 2022.

Bowman, S. R. et al. Measuring progress on scalable oversight for large language models. In *arXiv preprint arXiv:2211.03540*, 2022.

Bursuc, S., Ehrenborg, T., Lin, S., Astefanoaei, L., Chiosa, I. E., Kukovec, J., Singh, A., Butterley, O., Bizid, A., Dougherty, Q., Zhao, M., Tan, M., and Tegmark, M. A benchmark for vericoding: formally verified program synthesis, 2025. URL https://arxiv.org/abs/2509.22908.

Chan, Z. C. et al. Chateval: Towards better llm-based evaluators through multi-agent debate, 2024. URL https://arxiv.org/abs/2308.07201.

Chen, E., Gulati, A., Miranda, B., Tang, Z., and Koyejo, S. Rethinking LLM judges: Chain-of-thought and multi-step pipelines for math grading. In *ICLR 2026 Workshop on Logical Reasoning of Large Language Models*, 2026. URL https://openreview.net/forum?id=vdXPorr099.

Chen, M. et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Cheng, C.-A., Nie, A., and Swaminathan, A. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms, 2024. URL https://arxiv.org/abs/2406.16218.

Denisov-Blanch, Y., Kazdan, J., Chudnovsky, J., Schaeffer, R., Guan, S., Adeshina, S., and Koyejo, S. Consensus is not verification: Why crowd wisdom strategies fail for LLM truthfulness, 2026. URL https://arxiv.org/abs/2603.06612.

Ding, Z. et al. Citations and trust in LLM-generated responses. In *AAAI*, 2025.

Kenton, . et al. Scalable oversight via debate and consultancy. arXiv preprint, 2024. Add exact venue/ID if available.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. Dspy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=ICLR_2024_DSPy.

Kocmi, T. et al. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks, 2024. URL https://arxiv.org/abs/2406.18403.

Li, S. et al. Can you trust llm judgments? reliability of llm-as-a-judge, 2024. URL https://arxiv.org/abs/2412.12509.

Liu, . et al. Safe: Verifying every intermediate step in lean 4 with the formalstep benchmark. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

Liu, Y. et al. G-eval: Nlg evaluation using gpt-4 with better human alignment, 2023. URL https://arxiv.org/abs/2303.16634.

Ma, W. et al. Hypocompass: Using LLMs as teachable agents for debugging. In *International Conference on Artificial Intelligence in Education (AIED)*, 2024.

Miranda, B. Asking the right question: Formal methods as scalable oversight. https://brando90.github.io/brandomiranda/2026/04/22/formal-methods-scalable-oversight.html, April 2026. Blog post.

Miranda, B., Zhou, Z., Nie, A., Obbad, E., Aniva, L., Fronsdal, K., Kirk, W., Soylu, D., Yu, A., Li, Y., and Koyejo, S. Veribench: End-to-end formal verification benchmark for ai code generation in lean 4. In *ICML 2025 Workshop on AI for Math (AI4Math)*, 2025. URL https://openreview.net/forum?id=rWkGFmnSNl. Poster.

Naik, A. et al. TiCoder: LLM-based test-driven interactive code generation. *IEEE Transactions on Software Engineering*, 2024.

Nie, F., Liu, K. Z., Wang, Z., Sun, R., Liu, W., Shi, W., Yao, H., Zhang, L., Ng, A. Y., Zou, J., Koyejo, S., Choi, Y., Liang, P., and Muennighoff, N. Uq: Assessing language models on unsolved questions, 2025a. URL https://arxiv.org/abs/2508.17580.

Nie, F., Liu, K. Z., Wang, Z., Sun, R., Liu, W., Shi, W., Yao, H., Zhang, L., Ng, A. Y., Zou, J., Koyejo, S., Choi, Y., Liang, P., and Muennighoff, N. Uq: Assessing language models on unsolved questions, 2025b. URL https://arxiv.org/abs/2508.17580.

Ospanov, . and Yousefzadeh, . Apollo: Recursive repair with lean feedback boosts minif2f proof success. arXiv preprint, 2023. Add exact arXiv ID if available.

Rice, H. G. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. doi: 10.2307/1990888. URL https://doi.org/10.2307/1990888.

Robertson, . and Koyejo, S. Step-by-step but verified: Incentive-compatible, robust queries without ground truth. arXiv preprint, 2025. Add exact arXiv ID if available.

Salaudeen, . et al. Validity-centered evaluation for llm judges: A measurement framework. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

Salaudeen, O. and Hardt, M. Imagenot: A contrast with imagenet preserves model rankings, 2024. URL https://arxiv.org/abs/2404.02112.

Shen, J. et al. Symgen: Source-grounded, symbolically annotated generation for fast human verification. In *CLM Workshop*, 2024.

Sun, . et al. Clover: Closed-loop code–spec generation and smt-based verification. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

Tong, Y. and Zhang, C. Codejudge: Evaluating code generation with large language models, 2024. URL https://arxiv.org/abs/2410.02184.

VeriBench. Veribench: End-to-end formal verification benchmark for AI code generation in lean 4 (poster, ai4math @ icml 2025). https://icml.cc/virtual/2025/52376, 2025.

Wang, . et al. Theoremllama: Bootstrapping natural-language to lean 4 with iterative proof synthesis. arXiv preprint, 2024a. Add exact authors and arXiv ID if available.

Wang, S. et al. Investigating and designing for trust in AI-powered code generation tools. In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024b.

Wu, T. et al. Promptchainer: Chaining LLM prompts through visual programming. In *CHI Late-Breaking Work*, 2022.

Yan, J. et al. Intelliexplain: Natural-language explanations for interactive code generation. In *Proceedings of the 2024 Conference on Human Factors in Computing Systems (CHI) Workshop/Under Review*, 2024.

Zhang, L., Valentino, M., and Freitas, A. Beyond gold standards: Epistemic ensemble of llm judges for formal mathematical reasoning, 2025. URL https://arxiv.org/abs/2506.10903.

Zheng, . et al. Scrit: Self-evolving critics for scalable llm oversight. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

Zheng, L. et al. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL https://arxiv.org/abs/2306.05685.

Zhou, . et al. Don't trust, verify: Autoformalizing math solutions in isabelle/hol improves robustness. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

Zhou, J. P., Arnold, S. M. R., Ding, N., Weinberger, K. Q., Hua, N., and Sha, F. Graders should cheat: privileged information enables expert-level automated evaluations, 2025. URL https://arxiv.org/abs/2502.10961.

Zhuo, T. Ice-score: Llm-as-a-judge for code without ground truth outputs. arXiv preprint, 2024. arXiv ID not specified in source; update if known.

A. Related Work (Cont.)

Autoformalization. Complementing this line of work, Wang et al. (2024a) present THEOREMLLAMA, a framework aimed at enhancing LLM translation into Lean 4. Drawing on over 100K proof examples from the Mathlib4 library, TheoremLlama employs a novel natural-language-to-formal-language (NL-FL) bootstrapping strategy and iterative proof synthesis. This enables the reuse of verified examples as templates for future translations. The framework achieves 36.48% and 33.61% accuracy on the MiniF2F-Valid and MiniF2F-Test benchmarks, respectively—surpassing GPT-4 by more than ten percentage points on both.

Techniques for Code Verification. IMPROVER (Ahuja et al., 2024) introduces a Lean-aware Chain-of-States prompting loop that integrates retrieval, best-of-n sampling, and iterative correction to rewrite formal proofs with improved properties. By optimizing for metrics such as brevity and readability, ImProver reduces the number of tactics by half, doubles proof readability, and boosts theorem prover acceptance rates by over 80%. In addition, CLOVER (Sun et al., 2024) implements a closed-loop pipeline in which an LLM first generates code, docstrings, and formal annotations, then uses reconstruction-based prompting to enforce consistency across these outputs, and finally applies SMT-based verification to validate correctness. Evaluated on the CloverBench suite of Dafny programs, Clover accepts 87% of correct solutions, rejects 100% of flawed ones, and even uncovers bugs in human-written code—demonstrating the power of hybrid generation-verification pipelines. In a complementary direction, Zhou et al. (2025) improve general-purpose LLM-based graders by augmenting them with "privileged" information such as gold-standard solutions, grading rubrics, and detailed annotations. When necessary, the system provides targeted hints back to candidate models. This approach achieves grading performance on par with or exceeding that of specialized systems—and even expert humans—on difficult programming benchmarks.

Agentic Frameworks and Tools for Code Verification. TRACE (Cheng et al., 2024) proposes *generative optimization*, tuning entire computational workflows—including code, prompts, tool calls, and error signals—by treating execution traces as gradients in the OPTO framework. With a PyTorch-like API and the LLM-based optimizer OptoPrime, it supports diverse tasks such as prompt tuning, debugging, and robot control, rivaling specialized optimizers. Building on modular composition, DSPY (Khattab et al., 2024) treats LLM calls as declarative modules in a computational graph. Users define concise input–output signatures, and DSPy's compiler automatically bootstraps or fine-tunes pipelines using built-in "teleprompters." This enables a few-line programs to outperform expert-crafted prompts in math, QA, and agent workflows. Extending agentic capabilities to formal reasoning, PANTOGRAPH (Aniva et al., 2025) offers a programmatic interface to Lean 4 with support for advanced proof search. It exposes internal proof states and tactics for integration with learning agents, replacing human-facing interfaces with API-level control.

A.1. Related Work: Scalable Oversight

Scalable Oversight with LLM Judges Researchers have proposed multi-agent oversight frameworks such as debate and consultancy, where weaker LLMs act as judges supervising stronger agents (Kenton et al., 2024; Chan et al., 2024). Debate, in which two models argue opposing sides before a judge, consistently improves truth-tracking compared to single-pass QA. Kenton et al. (2024) demonstrate that debate reduces the likelihood of judges being misled in math and code tasks, highlighting it as a scalable oversight mechanism.

Self-Improving Critics. In the absence of stronger supervisors, models can bootstrap judgment skill through self-critique. The Self-Evolving Critic (SCRIT) framework trains LLMs to generate critiques of flawed outputs, validate corrections, and iteratively refine their ability to spot errors. Zheng et al. (2024) report improvements of over 10% on critique benchmarks, showing that self-critique fosters scalable, autonomous oversight.

Certifying Judges. Naïve LLM judges are often inconsistent, rewarding stylistic cues over correctness (Kocmi et al., 2024; Li et al., 2024).

Validity-Centered Evaluation. Certifying judges requires not only behavioral properties but also principled validity evidence. Salaudeen et al. (2025) introduce a framework that distinguishes between measurement, claims, and evidence, grounded in psychometrics. They emphasize construct, content, and criterion validity, and systematically enumerate threats to validity. This framework provides a blueprint for reporting and interpreting judge certification results, complementing identity/monotonicity checks with broader validity cards.

Robust Evaluation without Ground Truth. Standard judge queries can be brittle in the absence of ground truth, particularly under adversarial manipulation (Zhuo, 2024; Tong & Zhang, 2024). Robertson & Koyejo (2025) model the overseer as an agent and propose using information-theoretic, incentive-compatible queries instead of direct ratings. They introduce bounded f-divergence measures such as TVD-MI, which exhibit polynomial robustness to adversarial strategies, unlike unbounded KL-based criteria. This work highlights a pathway toward robust, certifiable judges when explicit ground truth is unavailable.

Single-judge certification vs. peer-prediction consensus. Information-elicitation approaches such as TVD-MI (Robertson & Koyejo, 2025) certify trust by asking multiple judges whether their outputs share information about the same source—fundamentally a consensus-based protocol over a judge population. Property certification is complementary: identity and monotonicity properties evaluate a single judge in isolation, with no agent-population dependency. This makes property certification deployable in settings where consensus is unavailable (single-judge inference, novel domains, sparse model panels), and our calibration protocol amortizes the human-annotation cost that consensus mechanisms also incur.

Consensus aggregation vs. property certification. Denisov-Blanch et al. (2026) stress-test five inference-time aggregation strategies—majority vote, highest-confidence selection, confidence- and prediction-weighted votes, and the Surprisingly Popular algorithm—across multiple benchmarks (HLE, BoolQ, Com2Sense, and Predict-the-Future) and open-source models spanning roughly 4B to 235B parameters; even at $25\times$ the cost of single-sample inference no aggregator consistently beats a single sample in verifier-absent domains, and several amplify shared misconceptions. They trace the failure to structurally correlated LLM errors: in MATH plurality-wrong cases, the most common incorrect answer accounts for 65–87% of wrong responses, and in an out-of-distribution random-ASCII control with no ground truth, different models still agree at Cohen's κ as high as ≈ 0.35, indicating shared inductive biases rather than shared knowledge. A central diagnosis is that self-reported confidence and consensus track *what the crowd will say* more reliably than *what is true*, separating social prediction from truth verification, and the paper concludes that inference-time scaling improves truthfulness only when an external verifier is available. Our work sits on the verifier-present side of that boundary: instead of aggregating across a judge population, we calibrate a single judge against falsifiable properties (P1–P4) over Lean 4-checked gold artifacts and explicitly constructed bug/specification ladders, yielding a Trust Index with no inter-judge consensus dependency. For oracle-free extensions, TVD-MI-style cross-judge information should be used as a robustness diagnostic rather than as a truth proxy, side-stepping the consensus-as-truth failure mode they document; conversely, their negative results sharpen the case for property-based, single-judge certification whenever a verifier or calibrated reference exists.

Formal Methods as Oversight. Proof assistants provide binary, kernel-level guarantees. Zhou et al. (2024) autoformalize math solutions into Isabelle/HOL and show that filtering via proof checking outperforms majority voting by 12% on GSM8K. Liu et al. (2025) extend this approach with SAFE, which verifies every intermediate step in Lean 4 and introduces the FormalStep benchmark, reducing hallucinations and enabling fine-grained oversight.

Agentic Iterative Pipelines. Ospanov & Yousefzadeh (2023) integrate Lean feedback into recursive repair cycles, boosting general-purpose LLM proof success from 3–7% to over 40% on miniF2F, and achieving 75% with fewer than 1000 samples. Miranda et al. (2025) extend this paradigm to program verification, showing that self-debugging agents quadruple success compared to one-shot prompting. Incorporating an LLM judge improved completeness on security tasks but required calibration to avoid regressions on simpler cases.

Benchmarks for Judge Reliability. Benchmarks such as VeriBench and FormalStep quantify judge reliability across compilation, proof success, and specification quality. Miranda et al. (2025) report normalized specification scores of 0.470 for one-shot prompting versus 0.615 for self-debugging agents. Outside formal mathematics, MT-Bench and G-Eval provide complementary judge-evaluation settings using pairwise preferences and rubric-guided scoring, underscoring the need for calibration and validity reporting (Zheng et al., 2023; Liu et al., 2023). Together with validity frameworks and robust information-theoretic scoring, these benchmarks advance the agenda of certifying LLM judges for formal verification.

Evaluating Autoformalization with Reference-Free Judges. In a complementary line of work, Zhang et al. (2025) propose an ensemble of LLM judges for the autoformalization task, using a fine-grained taxonomy to score formalizations without a gold-standard reference. The primary strength of this approach is its reference-free nature, offering a scalable evaluation pathway when gold standards are unavailable or prohibitively expensive to create. However, the work's motivation for abandoning gold-standard comparison is based on limited evidence: a small, non-random sample of "potentially incorrect" formalizations pre-selected by an LLM, whose own reliability is uncertified. Furthermore, the framework validates its judge by correlating with human preference scores—a subjective proxy—rather than the more objective, machine-checkable goal of logical equivalence to a known-correct specification. Critically, even if minor errors exist in gold-standard datasets, recent work has shown that such noise rarely alters the relative ranking of models, as errors are typically correlated across systems (Salaudeen & Hardt, 2024). This suggests that a small number of flawed examples does not invalidate the utility of an entire benchmark for comparative evaluation. Our work, in contrast, proceeds from the basis that gold standards are sufficiently reliable for ranking and focuses on certifying a judge's logical consistency against them – which is the desirable gold metric to be using.

A.2. HCI for Solving Scalable Oversight

From more evaluation to *human-centered* oversight. HCI reframes scalable oversight as a *calibration* problem: design interactions that let people verify, steer, and place *appropriate* trust in LLM outputs rather than maximizing raw trust (Bach et al., 2022). Interface choices (e.g., citations, explanations, confidence cues) can substantially shift user trust—sometimes in misleading ways—so the goal is to expose process and embed lightweight verification instead of presenting one-shot answers (Ding et al., 2025). User studies in AI pair-programming similarly surface needs to communicate model performance, provide control over output style/constraints, and expose uncertainty to avoid over-reliance (Wang et al., 2024b).

Hybrid human↔AI evaluation. HCI work operationalizes scalable evaluation by combining expert/lay judgments with automated judges. Humans annotate failures or verify slices of output; systems then train or condition LLM-based "reference evaluators" on those signals to scale up assessment while staying anchored to what users value (accuracy, safety, usefulness). This human-in-the-loop pattern complements LLM-as-judge approaches—whose reliability varies by task and setup (Kocmi et al., 2024; Li et al., 2024)—and is especially relevant when pass@k or single-shot metrics are brittle.

Interfaces that *operationalize* oversight. HCI systems instantiate oversight as interaction loops that make correctness checkable *during* generation: (i) *test-driven code generation* (TiCoder) elicits yes/no judgments about candidate unit tests before showing code; a few interactions reduced cognitive effort and lifted pass@1 by ∼45.7% on coding benchmarks (Naik et al., 2024); (ii) *explain→correct loops* (IntelliExplain) pair each snippet with a natural-language rationale that users can critique, improving novice task success by ∼11–25% (Yan et al., 2024); (iii) *pair debugging* (HypoCompass) has humans hypothesize faults while the LLM attempts fixes, yielding ∼12% learning gains (Ma et al., 2024); (iv) *multi-step pipelines* (PromptChainer) expose intermediate steps (e.g., tests → verification → proof attempt), enabling user intervention where failures arise (Wu et al., 2022); and (v) *source-grounded verification* (SymGen) highlights claim-level spans backed by exact sources and leaves unsupported text unhighlighted, speeding verification by ∼20% and raising user confidence (Shen et al., 2024). Collectively, these patterns shift trust from fluency to *verifiable progress*.

Applying HCI patterns to formal specs/theorems. Formal kernels (Lean/Isabelle) already deliver binary checks; HCI contributes the *workflow glue* that helps users evaluate *which* theorem/spec to prove and whether it is complete. Two actionable patterns: (1) *counterfactual spec probes*—UI toggles that remove a postcondition or inject a known bug and display the impact on tests/judge score—help detect missing clauses or overfitting; and (2) *stepwise proof oversight*—surfacing intermediate goals and marking unverified suggestions—keeps the human in control of proof search. In our setting, these patterns interface naturally with a graded, semantics-sensitive evaluator.

A.3. Vericode

Bursuc et al. (2025) presumes well-formed formal specifications and evaluates end-to-end verification under those specs (in Dafny, Verus/Rust, or Lean) reporting verified pass rates. In practice, impactful problems often lack both complete specifications and ground-truth answers. Our VERIBENCH program targets this gap by challenging the *generation* of specifications (from code, tests and documentation) and then *grading* their quality prior to proof search using a Lean-aware LLM judge that we validate via falsifiable principles/properties (Identity, Monotonicity, and Prompt Stability/Invariance), addressing known pitfalls of LLM-as-a-judge such as bias and prompt sensitivity (Zheng et al., 2023; Kocmi et al., 2024; Li et al., 2024). However, our framework has only been tested with a judge with gold references files with specifications but we will explore metrics/methods that don't require this like TVD-MI in (Robertson & Koyejo, 2025).

Beyond spec-first settings, the Unsolved Questions (UQ) paradigm evaluates models on difficult, real-world problems that have neither ground-truth answers nor formal specifications; instead, it relies on validator pipelines and community verification to track progress without oracles (Nie et al., 2025a;b). Taken together, these lines of work shift evaluation from "solve what is already specified" to "specify, calibrate, then solve": vericoding provides end-to-end rigor under given specs, VERIBENCH adds specification synthesis with a calibrated semantic judge, and UQ extends evaluation to specless, answerless settings that require validator-assisted screening. Our contribution is a calibrated, semantics-sensitive judge that enables scalable oversight when the specification must be proposed rather than merely proved. Because specification generation often lacks a unique ground truth, we will extend our oracle-free property-calibration protocol with UQ-style validators, adding cycle-consistency tests (spec to code/docs) alongside our bug and ablation ladders, and enabling abstention when validators disagree to prioritize precision over coverage.

B. Discussion (Cont.)

Formal methods as scalable oversight. A central concern with LLM-assisted research is that models produce text faster than humans can audit, so trust in the output can erode even when the surface looks correct. The concern treats verification as a single unstructured activity: one human eye against a wall of generated tokens. With a proof assistant, the activity decomposes. The Lean kernel deterministically checks every step of a proof, so what the human must verify is not the proof but a much smaller artifact—the formal statement, its definitions, and its assumptions—and whether they faithfully express the informal claim. This is the structural move that makes oversight scalable: the verifier checks the answer, while the human checks the question. The premise is timely: autoformalization is improving, language-model-assisted proof search has become usable rather than a demonstration, and the surrounding tooling (`mathlib4`, Lean's metaprogramming layer, Pantograph, Aesop) has matured to the point where formal verification of nontrivial claims is no longer a niche enterprise. This view sharpens, rather than dissolves, the concerns motivating human oversight: shallow reading and unjustified confidence remain failure modes, but the target of human understanding shifts from hundreds of lines of generated argument to a Lean statement together with its definitions and assumptions—a much smaller object, but the one that actually encodes intent. Theoretical limits remain—Rice's theorem, the halting problem, and Gödelian incompleteness bound what any countable formal system can decide—but they constrain what the verifier can settle, not the central oversight claim that humans are best deployed on *asking the right question*. For an extended informal discussion of this view, see (Miranda, 2026).

Why Lean 4? Lean 4 is a full fledge programming language and lets VeriBench contain full runnable code and machine-checked proofs in the same dependently-typed language. Ahead-of-time compilation with the Lake toolchain produces fast native binaries, while first-class `Task` primitives, async I/O, and a thread-pool scheduler enable genuine concurrent programs inside the prover. Lean's C-level foreign-function interface (FFI) lets those binaries call out to high-performance libraries when needed. On the proof side, Lean ships a powerful metaprogramming system written in Lean itself, giving researchers a programming interface to access its internals. This led to the creation of tools such as Pantograph and Aesop. Finally, the community-maintained `mathlib4` and `std4` libraries supply thousands of reusable theorems and data structures, and they are expanding quickly thanks to an active Zulip and GitHub ecosystem. Lean also has a work-in-progress program verification library, CSLib, which has recently seen rapid growth. Lean 4 is backed by an unusually vibrant open-source community: hundreds of contributors refine `mathlib4` on GitHub each week, the public Zulip sees expert discussion around the clock, and even Fields-Medalist Terence Tao has chosen Lean to formalize portions of his current research—clear testimony to the ecosystem's accessibility and intellectual depth. Unlike the unit tests, fuzzers, and code sanitation pipelines common in industry—tools that sample inputs or rely on heuristics—Lean 4 supplies machine-checked proofs that a property holds for *all* executions. Its dependent type system can encode deep invariants (e.g., length-indexed arrays, bounded integers), so programs that violate them fail to compile, eliminating whole classes of bugs such as buffer overflows or integer wrap-around. Code, specification, and proof reside in the same file and are compiled by Lake into the shipped native binary, preventing the drift that arises when verification artifacts live outside the build. In short, Lean turns informal "best-effort" checks into formal, end-to-end guarantees without sacrificing performance or interoperability.

Lean 4's limitations. Lean's toolchain is still younger than Coq's or Isabelle's, making its standard libraries and automation smaller, and thus some formalizations demand extra groundwork. Lean's runtime has not been stress-tested at the scale of mainstream systems languages, meaning large-scale or safety-critical deployments may require additional vetting. Lean tactics could contain bugs and fail to typecheck on different environments. Acknowledging these gaps clarifies that VeriBench chooses Lean 4 for its unique unified programming-plus-proving model and modern automation hooks, not because it already matches the decades-old industrial maturity of older theorem provers. *Static vs. runtime caveat:* Lean's guarantees are *static* and specification-relative: they certify the Lean program meets

the stated properties, but they do not by themselves detect emergent runtime faults outside the model (e.g., I/O (Input/Output) failures, environment misconfiguration, FFI (Foreign Function Interface) unsoundness, resource exhaustion, or undefined behavior in linked C code). Such behaviors must either be modeled and proved, or mitigated with production safeguards (monitoring, sandboxing, input validation).

Lean 4 versus Dafny. Unlike Dafny, whose verifier translates each program into the Boogie intermediate language and then discharges first-order verification conditions with an SMT solver such as Z3, Lean 4 reasons natively in a dependently-typed calculus. Because Lean 4's types can mention run-time data and the very same source code is ahead-of-time compiled to a native binary via Lake, we can both state and prove value-indexed, higher-order properties (e.g., length-indexed vectorsand run the verified program itself—an end-to-end, fidelity that Dafny's SMT-centred, Boogie-to-Z3 workflow cannot natively match. This gives Lean the expressive power to specify and prove higher-order, data-dependent properties—precisely the kind of semantic guarantees VeriBench seeks to test—while still yielding runnable binaries compiled by the same toolchain. Dafny's SMT-centric workflow offers impressive push-button automation for imperative code but cannot natively encode the richer specifications (e.g., length-indexed vectors, algebraic invariants) that Lean handles directly.

Lean 4 versus TLA+. TLA+ excels at high-level specification of concurrent and distributed protocols, with correctness checked by the TLC model-checker and the TLAPS proof system that dispatches first-order obligations to external provers. However, TLA+ specifications are not executable programs; a separate implementation step is required, and state-space explosion can limit model-checking scalability. VeriBench instead needs a prover where the specification, proof, and runnable code live in the same language. Lean 4's dependently-typed core lets us capture fine-grained, data-dependent invariants and then compile the very same artifacts to fast native binaries—capabilities outside TLA+'s scope.

Lean 4 verus Verus. Verus is a verification framework for Rust programs using SMT solvers. A user state intermediate goals and an SMT backend fills in the intermediate steps. Compared to Verus, users have the option to use or not use SMT Solver-based tactics in Lean. This enables more fine-grained control and steering of the proof process. Verus also has a much larger kernel compared to Lean, and is more susceptible to soundness problem than Lean whose kernel is small.

C. Future Work (cont.)

While (Robertson & Koyejo, 2025) study demonstrates that total-variation-based mutual information (TVD-MI) provides robust, gaming-resistant evaluation without ground truth, many settings —such as VeriBench— do provide gold specifications in Lean 4. A promising direction is to develop hybrid protocols that combine TVD-MI's black-box robustness with supervised ground-truth checks (compile success, unit tests, post-condition proofs, and equivalence theorems). By calibrating MI scores against known gold outcomes, we can create composite metrics that preserve gaming-resistance while offering calibrated probabilities of correctness where supervision exists. This opens the door to semi-supervised label propagation, active selection of items to prove, and cross-system experiments in domains with heterogeneous supervision. Such an integration would extend Robertson's information-theoretic framework into a practical, specification-aware oversight system that leverages the best of both unsupervised robustness and supervised precision.

D. Why Judge Scores Are Specification-Relative

It is tempting to interpret a perfect score from our LLM judge as a guarantee of absolute program correctness. However, this is fundamentally impossible due to established limits from computability theory. The obstacle is not a lack of ingenuity in our benchmark design or judge prompting, but an established negative result.

Rice's Theorem. The core limitation stems from Rice's theorem, which states that any non-trivial, semantic property of programs in a Turing-complete language is undecidable.

> **Rice's Theorem (Informal):** Let $\mathcal{S}$ be any property of a program's input-output behavior (e.g., "the program always terminates" or "the output is always sorted"). If $\mathcal{S}$ is non-trivial (meaning some programs have it and some don't), then no algorithm can exist that decides for *every* program whether it satisfies $\mathcal{S}$.

Impact on the Gold Standard and the Judge. Rice's theorem directly implies that the gold reference files used by our judge are necessarily incomplete. If a recursively enumerable gold standard could capture *every* true semantic property of a program, one could use it to build a decision procedure that contradicts the theorem. Therefore, our gold standards, while rigorous, represent a curated but fundamentally finite set of important specifications.

This leads to a critical conclusion for our work: the judge's evaluation is **specification-relative**. A perfect score does

not certify universal semantic correctness; rather, it certifies perfect **conformance to our specific gold standard**. Acknowledging this limitation is essential for correctly interpreting the judge's scores and understanding the scope of our evaluation.

E. Judge Certification: Additional Plots

We report raw scatterplots used during certification of the LLM judge. These plots visualize the judge's response to controlled perturbations: (P1) identity/reflexivity, (P2) cumulative bug injection, and (P3) removal of specification items (tests/theorems). Figures A.1–A.3 show that scores drop as artifacts worsen; the bug plot is noisier with an early prompt, which motivated the simplified rubric used in the main experiments. P4 (prompt stability/invariance) is reported in the main text with dedicated aggregate plots.

Protocol variants. The scatterplots below are "sanity-check" runs with the simplest prompts to stress visible monotonic behavior. In the bug study (Fig. 10) the earlier, theorem-equivalence prompt produced a noisier trend; this prompted a simplified, rubric-first prompt that we use for all reported results in the main paper, yielding the stronger correlations cited in Fig. 2. For missing-specs (Fig. 9), both Claude 3.5 and 3.7 show clear negative trends; minor non-monotonicity in 3.7 disappears with the finalized rubric.

We note that, for the experiment shown in Figure 9, a more complex judge (Claude 3.7) does not satisfy the property that the LLM Judge Score should monotonically increase with the number of missing specs. Indeed, the Judge File Equivalence score was not monotonic, even for Claude 3.5, leading us to simplify the rubric used in our main paper.

F. Validation-Set Human Alignment Appendix

This appendix reports full visual diagnostics for Exp16/Exp17 under final robust settings (three validation label sets: pass1, pass2, avg).

F.1. Naive Codex-Style Blind Baseline

To contextualize TI-ranked judge performance, we evaluate a naive blind baseline on the same pair set ($n = 72$ overlap), using a single-pass, surface-form-oriented Codex-style rubric without property-calibration constraints (no monotonicity/stability calibration). The baseline tracks pass2 labels somewhat but remains substantially weaker and less consistent than the top TI-ranked judges, especially on pass1 and in absolute error.

Label Set	ρ_s	r	
pass1	0.171 ($p = 0.151$)	0.218 ($p = 0.0656$)	0.155 (p
pass2	0.388 ($p = 7.58 \times 10^{-4}$)	0.427 ($p = 1.87 \times 10^{-4}$)	0.343 ($p =$

Table 7: **Naive blind baseline vs. validation labels (0–5 scale).** ρ_s: Spearman, r: Pearson, τ: Kendall; MAE is absolute error on the 0–5 scale. This baseline is included as a sanity-check negative control and is not used for final ranking claims.

Quantity	Value	Notes
Human-human Spearman (pass1 vs pass2)	0.249	95% CI = [−0.015
Human-human Pearson (pass1 vs pass2)	0.375	95% CI = [0.076,
Human-human QWK (pass1 vs pass2)	0.372	95% CI = [0.073,
Human-human MAE (pass1 vs pass2)	0.778	On aligned $n = 72$, 0
Human-human MSE (pass1 vs pass2)	1.139	On aligned $n = 72$, 0
JS divergence (pass1 vs pass2)	0.081 bits	Symmetric histogram d
KL(pass1‖pass2)	0.415 bits	Asymmetric diver
Best validation-set Spearman (avg labels)	0.480	GPT-5.3-Codex P4_CH
Best validation-set Spearman (pass1 labels)	0.559	GPT-5.3-Codex P1_BA

Table 8: **Human-human agreement ceiling context.** Inter-pass human agreement is moderate, and inter-pass dispersion (MAE/divergence) is non-zero, so validation-set judge-human values near 0.5 are meaningful on this benchmark and should be interpreted with label-noise context.

F.2. Human-Human Agreement Ceiling Context

Validation-set judge-human Spearman values should be interpreted relative to the reliability of human labels themselves. Across Ethan pass1 vs pass2 on the aligned set ($n = 72$), inter-pass agreement is moderate (Spearman 0.249, Pearson 0.375, Kendall 0.221, QWK 0.372). Thus, validation-set judge-human Spearman values in the 0.48–0.56 range are non-trivial and should be interpreted as approaching the available annotation ceiling in this dataset rather than as "low absolute" values.

F.3. Cross-Model Validation Status (Full vs Preliminary)

To avoid over-claiming, we separate full-scale and preliminary evidence. Main conclusions in this paper use full-scale runs only (Exp16/Exp17: $n = 8$ judges, $n = 72$ aligned rows, pass1/pass2/avg label sets). Claude Sonnet certification is currently preliminary (3 files) and is included as directional evidence only, not as a core statistical pillar.

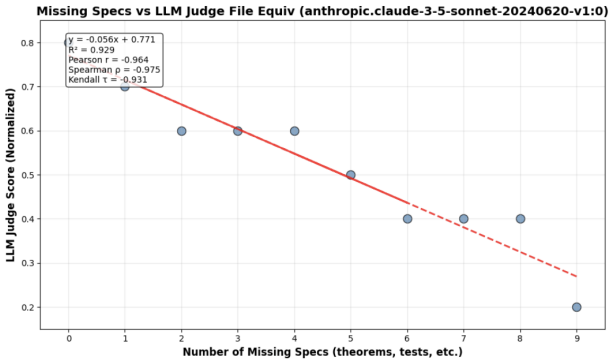
F.4. Leanstral Negative-Control Reaggregation (AM/GM/Variance-Aware)

To stress-test whether index choice alone can "rescue" a weak judge family, we re-aggregate the same Leanstral Exp16 outputs under three indices:

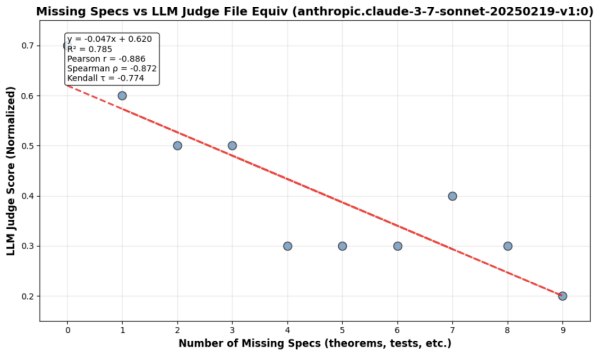
$$\mathrm{AM} = \text{cert_index_v2} \quad \text{(arithmetic baseline)},$$

$$\mathrm{GM} = \mathrm{mono}^{0.4} \cdot \mathrm{stab}^{0.6},$$

$$\mathrm{TI}_{\mathrm{var}} = \exp\Big(\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{\mathrm{VarCorr}}) + \tfrac{1}{3}\log(1-\widetilde{\mathrm{VarScore}})\Big),$$



(a) Claude 3.5 Sonnet v1: strong monotonic decrease ($y = -0.056x + 0.771$, $R^2 = 0.929$, Pearson $r = -0.964$, Spearman $\rho = -0.975$).



(b) Claude 3.7 Sonnet v1: clear negative trend with minor non-monotonicity ($y = -0.047x + 0.620$, $R^2 = 0.785$, Pearson $r = -0.886$, Spearman $\rho = -0.872$).

Figure 9: **(P3) Monotonicity w.r.t. missing specifications.** Candidate scores decrease as more theorems/tests are withheld. We retain both model variants to show robustness of the trend; 3.7's small non-monotonicity motivated the simplified rubric used in the main results.

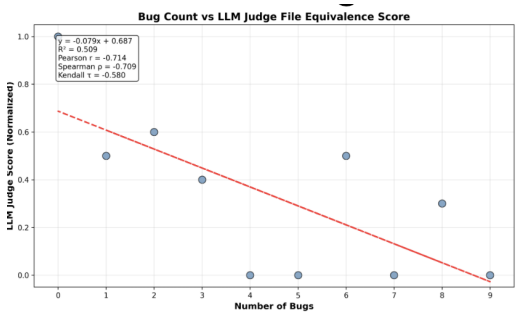


Figure 10: **(P2) Non-Monotonicity w.r.t. cumulative bugs (Claude 3.5 Sonnet).** Scores drop as more semantic bugs are injected (Pearson $r = -0.714$, Spearman $\rho = -0.709$, $R^2 = 0.509$); the higher noise with the initial "theorem-equivalence" prompt led us to simplify the rubric used for the final judge in the main paper.

where C_f is the mean per-file property-correlation quality term, and $\widetilde{\text{VarCorr}}$, $\widetilde{\text{VarScore}}$ are normalized variance penalties (lower is better before $1 - \cdot$). We report Spearman between index score and validation-set human Spearman across judges.

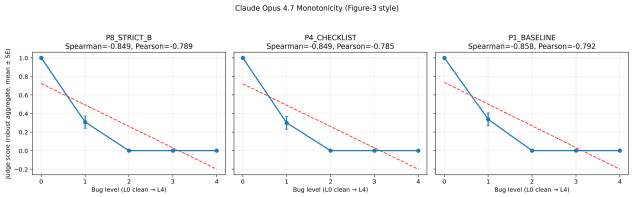
F.5. Failure Modes Found by Certification (Not Hidden)

The certification framework is useful precisely because it identifies failures: **(i) alpha-renaming sensitivity** is the dominant identity/invariance gap, and **(ii) local bug-ladder reversals** occur despite overall negative monotone trends. We therefore report both strengths and weaknesses, and include alpha-ablation diagnostics. On trend-v1 Spearman (TI $\rightarrow$ validation-set human Spearman), excluding alpha variants changes pass1 $0.810 \rightarrow 0.690$, pass2 $0.810 \rightarrow 0.714$, avg $0.881 \rightarrow 0.786$, showing that alpha cases materially affect measured robustness.

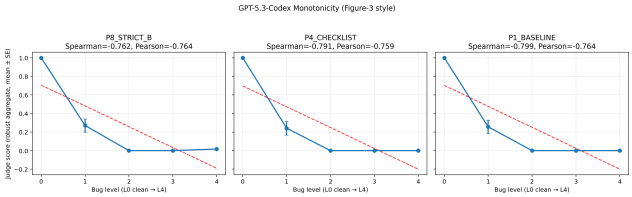
F.6. Newest Results (Latest Rerun at Appendix Bottom)

To reduce judge-call noise, we reran Exp16 with higher repeats and robust repeat aggregation: `JUDGE_REPEATS=9 REPEAT_AGG=median BOOTSTRAP_ITERS=10000 PERM_ITERS=20000`. This run writes to `results_exp16_final_robust_r9_median`.

We also generated a low-variance replot package from the same detail rows (`results_exp19_low_variance_replot`): a cleaner main monotonicity curve for core judges plus separate failure-mode plots for alpha sensitivity and local bug reversals. Reversal rates in this package are: GPT-5.3 P4/P1 $= 0.00$, GPT-5.3 P8 $= 0.167$, BAD_STRINGY $= 0.833$, GPT-4o variants $=$ 0.667–0.750, edit-distance/random $= 1.00$.

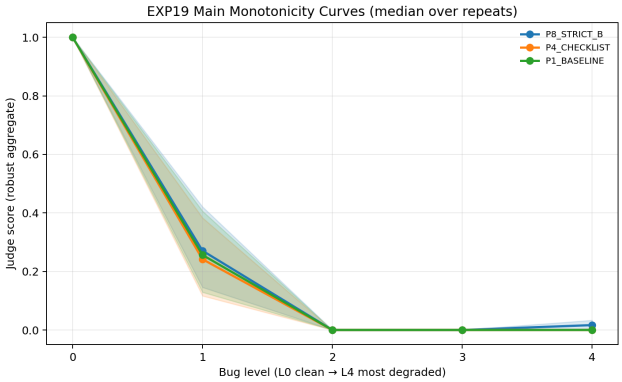


(a) Claude Opus 4.7 Figure-3-style monotonicity (mean ± SE).

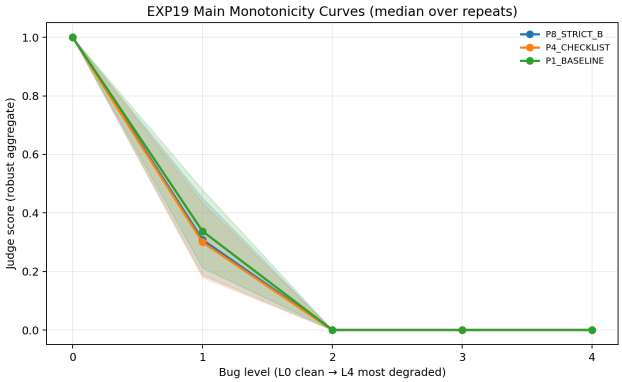


(b) GPT-5.3-Codex Figure-3-style monotonicity (mean ± SE).

Figure 11: **Model-matched Figure-3-style monotonicity diagnostics.** Both judge families show strong monotone score degradation as bug intensity increases, under the same low-variance plotting protocol.



(a) Codex low-variance monotonicity curves.



(b) Opus 4.7 low-variance monotonicity curves.

Figure 12: **Low-variance monotonicity trends across model families.** Both judge families show strong non-increasing behavior as bug intensity increases, under robust repeat aggregation.

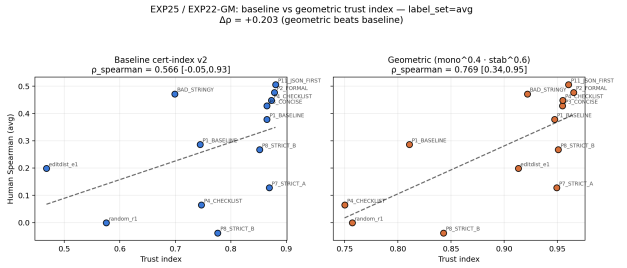
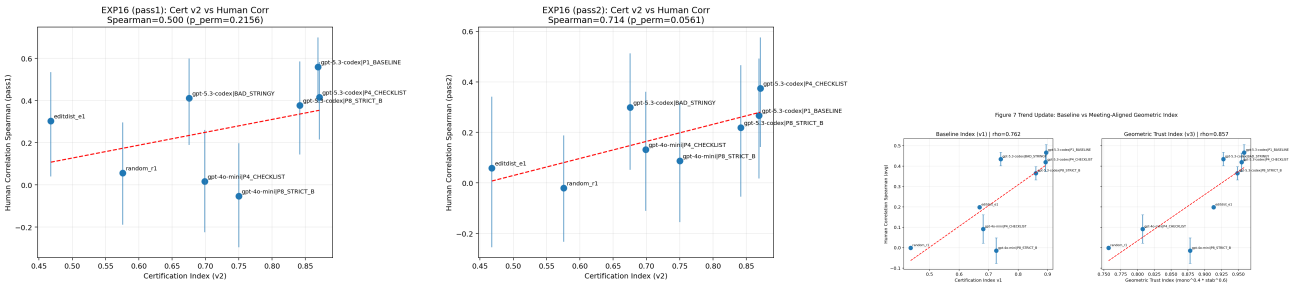


Figure 13: **Appendix: Expanded 13-judge panel trend (avg validation labels).** Geometric trust index retains strong alignment with human correlation at larger judge-panel size ($\rho = 0.769$); pass-specific values are pass1 $\rho = 0.764$, pass2 $\rho = 0.753$.

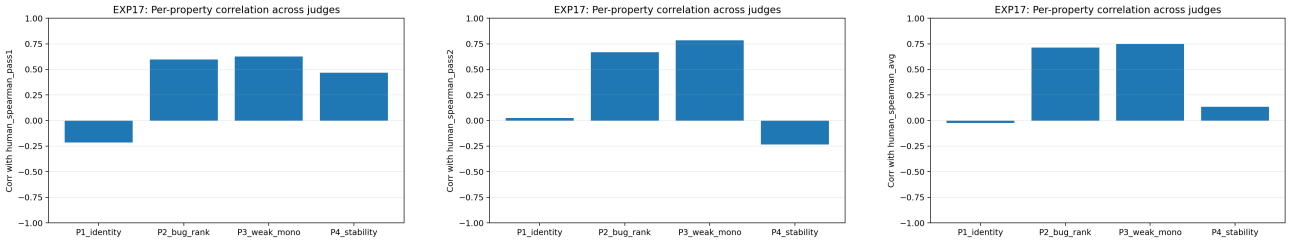
Judge Setup	Scale	In Main Claims	Role
GPT-5.3-Codex prompts (P8/P4/P1)	Full VeriBench pairs	Yes	Primary TI-ranked judges
GPT-5.3-Codex BAD_STRINGY	Full VeriBench pairs	Yes	Same-model stress baseline
Qwen3-Coder prompts (P8/P4/P1)	Full VeriBench pairs	Yes	Open-weights cross-family run
Qwen3-Coder BAD_STRINGY	Full VeriBench pairs	Yes	Open-weights stress baseline
GPT-4o-mini prompts (P8/P4)	Full VeriBench pairs	Yes	Cross-family baseline
Edit-distance / random baselines	Full VeriBench pairs	Yes	Non-LLM controls
Claude Sonnet certification run	3 files (prelim)	No	Directional cross-model check

Table 9: **Cross-model validation status.** This paper's inferential claims are grounded in full-scale runs; Claude results are explicitly preliminary and not used as primary evidence.



(a) Exp16 trend (pass1) (b) Exp16 trend (pass2) (c) Exp16 trend (avg)

Figure 14: **Judge-level TI-to-validation trend across all validation label sets (Exp16).** The strongest and most stable effect is for trend-v1 TI, with averaged labels giving the largest effect size.



(a) Exp17 per-property (pass1) (b) Exp17 per-property (pass2) (c) Exp17 per-property (avg)

Figure 15: **Property-wise validation-set human-alignment diagnostics (Exp17).** P3 weak monotonicity is consistently the strongest predictor across label sets; P2 is second strongest; P1/P4 are weaker as standalone predictors at this scale.

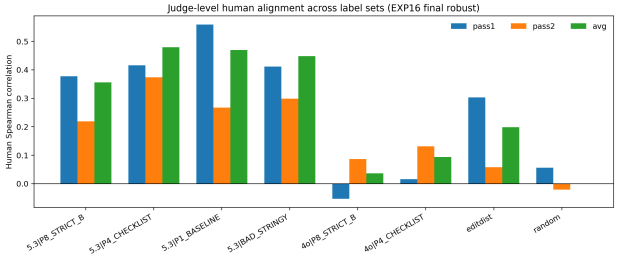


Figure 16: **Unplotted diagnostic added for this version:** judge-wise validation-set human Spearman across pass1/pass2/avg. GPT-5.3-Codex variants remain above weaker baselines on most label sets; pass-to-pass variation is visible and motivates reporting all three label sets.

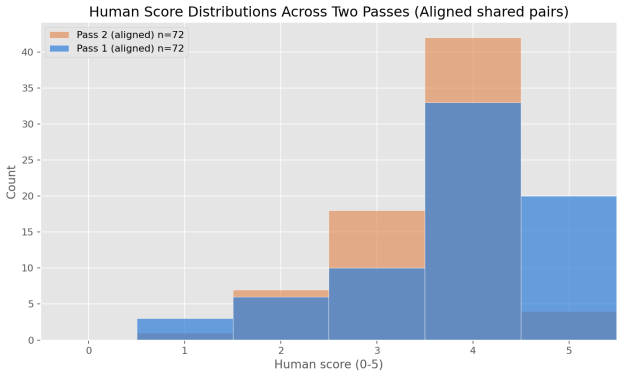
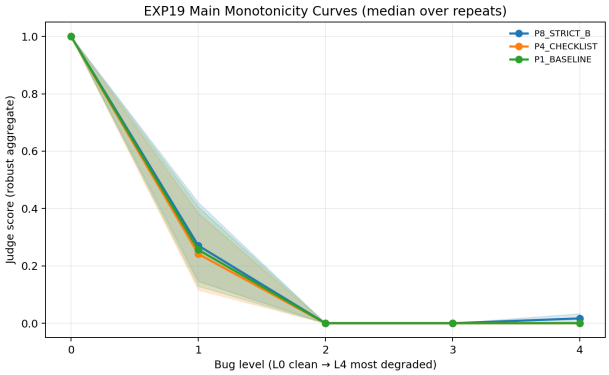
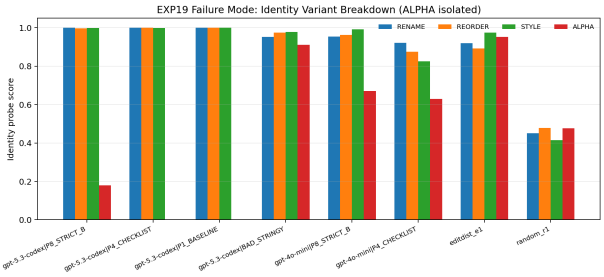


Figure 17: **Human-score dispersion across passes (aligned $n = 72$, 0–5 scale).** Overlaid pass1/pass2 histograms show visible distribution shift, consistent with the non-zero MAE/MSE and divergence statistics in Table 8.



(a) Main low-variance monotonicity curves (core GPT-5.3 judges, robust repeat aggregation).



(b) Failure-mode isolation: identity variant breakdown with ALPHA shown separately.

Figure 18: **EXP19 replot package: cleaner main trend + explicit failure separation.** The left panel is the "main story" plot with lower visual variance; the right panel isolates alpha-renaming sensitivity rather than mixing it into the primary monotonicity figure.

Label Set	**AM (v2)**	**GM**	**Variance-Aware** $\mathrm{TI}_{\mathrm{var}}$
val (pass1)	-0.5952	-0.1429	-0.1190
test (pass2)	-0.1905	-0.2619	-0.0476
avg	-0.4286	-0.2143	-0.0952

Table 10: **Leanstral remains low-alignment under multiple index constructions.** Re-aggregation improves over the arithmetic baseline in some splits, but all three settings remain near-zero or negative. This suggests the bottleneck is likely the underlying judge behavior for this ranking task (human-preference alignment), not only index form. One plausible hypothesis is that Leanstral is stronger for Lean proof editing/generation than for calibrated pairwise judging under this rubric; we treat this as a testable conjecture, not a final causal claim.

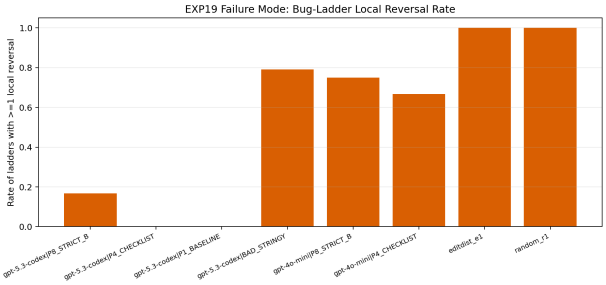


Figure 19: **EXP19 failure-mode plot: local bug-ladder reversal rates by judge.** This explicitly surfaces where monotonicity fails locally, separating robustness diagnostics from the primary trend plot.

Label Set	**v1 Spearman**	**v2 Spearman**
pass1	0.762 ($p = 0.0372$, CI $[-0.139, 1.000]$)	0.524 ($p = 0.1959$, CI $[-0.400, 0.975]$)
pass2	0.810 ($p = 0.0222$, CI $[0.291, 1.000]$)	0.571 ($p = 0.1477$, CI $[-0.231, 0.975]$)
avg	0.762 ($p = 0.0402$, CI $[0.103, 1.000]$)	0.476 ($p = 0.2426$, CI $[-0.425, 1.000]$)

Table 11: **Latest Exp16 rerun (R9 + median aggregation).** Increasing repeats and using median aggregation preserves the v1 trend signal while v2 remains more sensitive at this $n_{\text{judges}} = 8$ scale.